Sprint Report 4

Team Plugged In
David Jarman, Marshall Gaucher, Kevin Worner
Project STBMF
January 16th – February 6th 2013

Initial Sprint Backlog

This sprint was split between the Game Server and the Virtual Set Top Box. Team member Kevin and David worked on the server while Marshall worked on the vSTB. During the sprint planning meeting, several goals and expectations were set: A demo to show Roger, unit and regression testing discussion, and to have a code review by the end of the sprint.

Server

1. Forward CommandLink message to proper game instance
2. Refactoring and restructure server
   2.1. Pull out GUI connection from Information Exchange
   2.2. Refactor Information Exchange to have more descriptive name
   2.3. Implement methods for database access
   2.4. Move all networking aspects out of MPGameServer project and into the communication project
   2.5. Transfer data manipulation methods from DataHandler to Utility
   2.6. Restructure ConnectionManager (GUI)
3. Create a demo message between the server and client
   3.1. Initiate message from server
   3.2. Parse message on vSTB
   3.3. Display results
   3.4. Send reply to server
4. Database
   4.1. Determine what data is required to be stored
   4.2. Design schema/tables/relations
   4.3. Research entity frameworks for access to database
   4.4. Update database from code
   4.5. Add table for Logging
5. Game interface
   5.1. Create serializable class to hold full game states
   5.2. Design method definitions for games to implement
   5.3. Research loading DLL's at runtime
   5.4. Research calling methods on DLL's at runtime
   5.5. Research loading resources from DLL's
6. Update message classes to match current protocol definition document
7. TCP network interface
   7.1. Convert to Reactive Extensions (Rx)
   7.2. Accept, receive, and store messages
   7.3. Design message passing system to other components
8. Update Logger class to write to the database
9. Research cross process communication

Virtual Set Top Box

1. Game Model

1.1. Refactor model to create non selectable buttons to preserve movement logic of game
2. Game Piece
3. Game View
4. Game Base View
   4.1. Modify existing file system view/model classes
   4.2. Connect vSTB to vAPMax echo stubbed response app
   4.3. Request/response initial game list
   4.4. Modify stub messages for actual checkers resource files
   4.5. Refactor file system view/model class for sanity check of resource delivery
   4.6. Admin mode open all delivered resource files accordingly
   4.7. User mode will allow selection of new game or previous game from resource files to load vSTB game not raw data file
   4.8. Ensure load of response resource files and messages can be removed
5. Game statistics
6. Game menu
7. Create checkers game as a proof of concept
8. Refactor existing STB game view/model architecture to meet STBMF requirements
9. Quick button tool/status bar
10. Wireshark dissector research
11. Prove out and implement QUiLoader
12. Research dynamic UI
13. vAPMax stubbed message app
14. Research QT unit testing
15. FileSystemWatcher

Work Carried to Next Sprint

Virtual Set Top Box

All the main classes that make up the vstb multiplayer environment were completed this sprint. This provides the framework to refactor the classes as determined by communication of live environment. The Gameview class needs to be slightly modify to essentially mimic the movements of a stb remote so I have designed a simple remote to emit the signals(e.g. up,down,left,right,menu,info) to the view. The communication layer needs additional message handling logic to handle larger UDP messages (game resource files) sent down the wire. Until we can validate the integrity of a game resource file, I will simply modify the Gameview, Gamemodel, and Gamepiece class to read from local game resource files that are valid. This provides a point of reference for how the GUI should behave and look when the communication layer is stable.

Server

All server related tasks were completed this sprint allowing us to focus on the rest of the project in following sprints, including implementing Checkers logic, Checkers A.I., unit testing, and creating an user interface to add games.

Issues

Virtual Set Top Box

1. Gameview/ Gamemodel / Gamepiece
    1.1. GUI or NUI
    The user's interaction with the vSTB app. There needs to be a clear distinction if it is a GUI or a NUI for our proof of concept environment, as the movement logic is simplified drastically in a NUI. The NUI vSTB mimics a touch screen device where the user naturally interacts with the app. Additionally, there is no need design special logic to handle board games and card games because the movement it is handled by the NUI and it only cares about user's selection. Creating a GUI to fudge movement across all game models creates a poor user interface that ultimately isn't consistent. The lack of consistency reduces the scalability and modularity of the vSTB multiplayer environment since there has to be special movement logic designed per game type. Adding a vSTB remote to the interface reduces the overall screen real estate of a game app and reduces the simplicity of the user's interface.
    1.2. Timeline of Implementation
    Since our project is a proof of concept there needs to be clear vision for the ultimate purpose of this project. The reason is because current STB, apmax, and server do not support our communication logic. The underlying goal of this project is to prove bidirectional communication across the entire iptv system within the scope of STB games, since the current iptv architecture is limited in this respect. Designing a proof of concept to a system that needs to be redesigned to actually handle this communication doesn't make sense. More importantly if this project were to go into production, the raw time required to implement it across existing network components would age the vSTB system; ultimately reducing the scalability and modularity of this project.
    1.3. STB Hardware Evolution
    There needs to be a step back approach to the overall vision of this project as current STB hardware is extremely dated. I realize from a business perspective that the vSTB needs to work on all current STB AMINO products but this is a proof of concept and needs to remain that way to support future STB AMINO products.
1. Database
    1.1. When researching ways to access a database using .NET and C#, the first option we decided upon was the idea of datasets, which did not auto generate entity classes. This posed a large problem in terms of the server organization and flexibility, which is so important at this stage of implementation. After creating a few methods to prove we could access the database, we decide to use the Linq to Entities framework. Some of the features we found particularly

tantalizing was the ability to use Linq to generating SQL statements, viewing the database as an object graph, meaning that the details of the data store are hidden, and being able to update our in memory definition easily as we change the definitions at the database level.

1.2. One user story that was given was the ability to change the location, username, and password of the database within the server. This caused some confusion and clarification was needed to distinguish whether this meant at runtime or compile time, and the exact details of how it should be done to make it easiest for those maintaining the server.

2. Serialization

2.1. Some classes are serializable, meaning that we can turn an object into a byte array and store it somewhere with little coding effort. In a first attempt to serialize some objects, we found that the .NET serializer created unnecessarily large byte arrays. After some discussion, we decided to use Google's open source serialization method, called Protocol Buffers. We observed that same object we had previously serialized using .NET used about half the memory when serialized under Protocol Buffers.

Next Sprint

Virtual Set Top Box

The Gameview, Gamemodel, and Gamepiece need to be slightly modified to read from local resource files. Implement Unit Testing for model/ view classes.

Server

In the next sprint, we will finish up the bare bones of the server so that it can be run as a service. There is work to be done in terms of messaging yet. We need to split large messages into multiple messages. This is to accommodate the UDP message passing between the APMax and STBs. We will also begin implementing the logic of checkers. After we have the logic in place, we can begin work on the artificial intelligence aspect of checkers.